

ML-KEM 解封装实现的时间侧信道泄漏检测

姓名：寇得一，学号：2023211545

摘要：ML-KEM 是 NIST 于 2024 年发布的后量子密钥封装标准,其算法安全建立在 Module-LWE 困难性与 Fujisaki-Okamoto 变换之上,但算法层面的安全性不能自动推出软件实现在执行时间上不泄漏信息。本研究以 `pqcrypto` 库的 ML-KEM-512、ML-KEM-768 和 ML-KEM-1024 解封装实现为对象,构造有效密文与四类同长度无效密文,采集端到端解封装时间,并将 Welch 检验、KS 检验、Cohen 效应量、分组机器学习分类、固定延迟正对照和延迟灵敏度扫描结合为联合判定流程,以区分算法规范安全与具体实现安全。在覆盖三个参数集和四种无效密文策略的 60 次主实验中,被测实现未表现出稳定可区分的时间侧信道泄漏;在 macOS arm64 平台两种后端的补充实验中得到一致结果;延迟扫描确定约 2.4–3.5 μs 的 80% 功效检测边界。在当前软件计时条件和输入模型下,未检测到被测 ML-KEM 解封装实现存在可稳定利用的端到端时间泄漏;该结论属于实现级筛查,不构成形式化侧信道安全证明。项目代码与实验数据已开放于 GitHub: <https://github.com/AndrewAccuracy/PostQuantum>。

关键词：后量子密码; ML-KEM; 时间侧信道; 实现安全; 泄漏检测

1 引言

1.1 研究背景

公钥密码体制是现代网络安全基础设施的核心组成部分,广泛用于密钥协商、身份认证、软件签名和安全通信协议。RSA、Diffie-Hellman 和椭圆曲线密码等经典方案的安全性主要建立在大整数分解和离散对数问题的计算困难性之上。Shor 在 1994 年提出的量子算法表明,具备足够规模和纠错能力的量子计算机能够在多项式时间内求解这两类问题^[1]。因此,即便大规模通用量子计算机尚未实用化,密码系统也需要提前迁移到能够抵抗量子攻击的后量子密码算法。

这种迁移需求具有明显的前置性。一方面,公钥密码算法深嵌在 TLS、VPN、软件供应链、身份认证和长期数据保护等场景中,协议升级、软硬件适配和互操作测试都需要较长周期;另一方面,部分长期保密数据可能面临“先收集、后解密”风险,即攻击者现在保存加密通信内容,等待未来量子计算能力成熟后再尝试解密。因而,后量子密码的研究不只是算法竞赛意义上的理论问题,也直接关系到现有系统能否平滑完成密码迁移。

NIST 自 2016 年启动后量子密码标准化进程,并于 2024 年发布首批后量子密码标准。其中 FIPS 203 将 CRYSTALS-Kyber 标准化为 Module-Lattice-Based Key-Encapsulation Mechanism,即 ML-KEM^[2-3]。本研究依据 FIPS 203 正式发布版展开实验,不单独分析后续 errata 对实现行为的影响。ML-KEM 基于模格误差学习 (Module Learning With Errors, MLWE) 问题,提供 ML-KEM-512、ML-KEM-768 和 ML-KEM-1024 三组参数,分别对应不同安全级别和性能开销。

因此,讨论 ML-KEM 的部署安全需要区分两个层次。第一层是算法规范的计算安全性:ML-KEM 的密钥隐藏能力来自 Module-LWE 困难性,抗量子能力来自当前不存在类似 Shor 算法那样可高效求解该问题的量子算法。第二层是具体实现的侧信道安全性:即使底层困难问题仍然成立,程序执行过程中的分支、内存访问、算术指令延迟或错误处理路径也可能泄露规范接口之外的信息。本研究的核心问题正位于第二层,即在标准算法已经具备理论安全依据的前提下,检查常见软件实现是否表现出可稳定观测的时间差异。

在实际协议中,KEM 通常用于在不安全信道上协商共享密钥,再由对称密码承担后续数据加密任务。与传统公钥加密相比,KEM 的接口更贴近现代混合密钥交换协议,也更适合作为后量子迁移中的基础组件。ML-KEM 的三个参数集在安全级别、密钥长度、密文长度和运行开销之间提供不同折中;因此,仅考察单一参数集不足以反映部署时可能遇到的实现行为,有必要在全参数集上进行横向测试。

然而,算法标准化解决的是规范层面的安全问题,并不能自动保证具体实现没有侧信道泄漏。Kocher 的时序攻击工作最早系统性说明了公钥密码操作的执行时间可能泄露私钥信息^[4];Bernstein 随后展示了缓存状态也可能形成可利用的时间差异^[5]。对于后量子 KEM 而言,解封装阶段同时接收攻击者可控密文并使用私钥,是实现安全分析中的重点环节。已有研究表明,CCA 安全格基 KEM 中的 Fujisaki-Okamoto 变换和重加密验证步骤可能成为功耗或电磁侧信道分析的重要目标^[6-7]。因此,对 ML-KEM 实现开展可复现的泄漏检测具有直接的工程意义。

从工程评估角度看,侧信道问题往往出现在规范与实现之间的缝隙中。规范可以要求解封装接口在验证失败时仍输出伪随机共享密钥,却很难完全限定编译器优化、底层算术指令、内存层次结构和运行时环境带来的微小时间差异。对于软件库使用者而言,最先需要回答的问题通常不是能否立即构造密钥恢复攻击,而是某一实现是否已经表现出可稳定观测的异常时间行为。本研究正是在这一层面上展开:把端到端解封装时间作为可观测信道,对有效密文与多类同长度无效密文进行可复现筛查。

1.2 研究目的与问题提出

本研究的目的,是面向 ML-KEM 解封装这一实现安全敏感环节,建立一套可复现、可量化、可解释的软件时间侧信道泄漏筛查方法。我们不以构造完整密钥恢复攻击为目标,也不试图证明某一实现具备形式化侧信道安全性,而是关注攻击者可控密文输入是否会使解封装过程表现出稳定、可检测、可进一步分析的执行时间差异。

我们选择解封装作为观测对象,主要基于两点考虑。首先,解封装接口同时处理私钥和外部输入密文,攻击者可以反复提交选择密文并观察返回行为,是 KEM 实现中最接近选择密文攻击模型的接口。其次,ML-KEM 的解封装过程包含解密、重加密、密文比较和失败路径密钥派生等步骤;这些步骤在规范上应避免泄露成功或失败路径差异,但在具体软件实现中仍可能受到分支、内存访问、算术指令延迟和异常处理路径的影响。因此,比较有效密文与无效密文的端到端时间分布,是发现实现级异常行为的一种直接入口。

围绕上述目的,主要回答以下研究问题:

- RQ1** 在 pqcrypto 库提供的 ML-KEM-512、ML-KEM-768 和 ML-KEM-1024 解封装实现中,有效密文与无效密文之间是否存在可由统计检验识别的执行时间差异?
- RQ2** 在多维时间统计特征下,机器学习分类器能否在未见过的基础密文组上稳定区分有效密文与无效密文?
- RQ3** 不同参数集和不同无效密文构造策略是否会改变泄漏检测结果?
- RQ4** 在软件计时噪声存在的条件下,检测管线能否识别已知人为时间信号,其灵敏度边界大致处于什么量级?
- RQ5** 当平台和底层 ML-KEM 实现发生变化时,补充实验是否仍给出与主实验一致的筛查结论?

通过上述问题,我们希望给出一个边界清晰的实现级判断:若检测到稳定信号,则说明被测实现需要进一步侧信道审计;若未检测到稳定信号,则只能说明在当前实验平台、输入模型和计时方法下没有发现可区分泄漏,而不能推出算法或实现的绝对侧信道安全。

上述问题的提出基于两个事实。第一,ML-KEM 已进入标准化和部署阶段,FIPS 203 对算法接口、参数和正确性给出规范,但实现仍需接受独立的侧信道评估^[2]。第二,传统测试向量泄漏评估(TVLA)强调通过统计检验发现两类输入之间的泄漏信号^[8],而机器学习分类器能够从多维特征组合中发现非线性或非均值差异。因此,我们将参数检验、非参数检验、效应量和机器学习判别能力结合起来,形成多证据交叉验证的实现级检测方法。

1.3 主要贡献

主要贡献如下:

1. 面向 ML-KEM 解封装实现,构建了一套可复现的软件时间侧信道泄漏筛查框架,覆盖 ML-KEM-512、ML-KEM-768 和 ML-KEM-1024 三个参数集,并支持多轮重复实验与自动化结果分析。
2. 设计了四类同长度无效密文输入模型,包括单比特扰动、字节翻转、伪随机密文和全零密文,用于从最小扰动、局部破坏、随机无效输入和极端结构化输入等角度考察解封装时间行为。
3. 结合统计显著性、效应量和机器学习可分性建立多证据泄漏判定方法,并通过分组交叉验证降低同一基础密文组带来的记忆性数据泄漏风险。
4. 引入固定延迟正对照和延迟灵敏度扫描,验证检测管线对已知时间信号的识别能力,从而提高负检测结论的可信度。
5. 基于 5 轮独立重复、3 个参数集和 4 种无效密文策略的 60 次完整实验,给出被测 pqcrypto ML-KEM 解封装实现的软件时间泄漏筛查结果,并明确说明该结果的适用边界。
6. 将 macOS arm64 平台上的 pqcrypto 和 liboqs 结果作为补充验证单独报告,而不与 Windows 主实验混合统计,从而区分主结论依据和跨平台、跨实现稳健性观察。

实验结果表明,在软件黑盒计时条件下,pqcrypto ML-KEM 解封装实现对有效密文与无效密文未表现出稳定可区分的时间差异;该负结论在约 2.4-3.5 μ s 的灵敏度边界内成立,并在 macOS arm64 平台上通过 pqcrypto 与 liboqs 两种后端获得跨平台和跨实现的初步验证。

本文其余部分安排如下:第 2 节回顾时间侧信道、格基 KEM 侧信道和机器学习辅助泄漏检测相关研究;第 3 节介绍 ML-KEM 参数集、统计检验与分组机器学习评估基础;第 4 节说明实验对象、威胁模型、无效密

文策略和判定规则；第 5 节给出实验结果与灵敏度分析；第 6 节讨论结果解释、局限性和可复现性；最后给出结束语。

2 相关工作

2.1 后量子标准化与 ML-KEM 实现评估

后量子密码标准化的核心目标，是在量子计算威胁下为现有公钥密码基础设施提供可替换的算法组件。NIST 标准化过程从安全性、性能、密钥与密文规模、实现复杂度和部署可行性等方面综合评估候选算法，最终将 CRYSTALS-Kyber 标准化为 ML-KEM^[2-3]。从部署角度看，标准文本给出了算法接口、参数集、编码方式和正确性要求，为不同实现之间的互操作奠定基础。

但是，标准化并不等价于所有实现都天然满足常数时间或侧信道安全要求。ML-KEM 的实现可能来自参考代码、优化汇编、语言绑定或第三方密码库，不同实现会受到编译器、处理器微架构和运行时系统的共同影响。即使两个实现遵循同一算法规范，其内存访问模式、算术指令选择和错误处理路径也可能存在差异。因此，在算法进入工程部署阶段之后，独立的实现级测试成为标准化工作之外的必要补充。

已有研究从不同角度评估了 Kyber/ML-KEM 的实现安全性。Kannwischer 等人比较了多个后量子密码算法在嵌入式处理器上的性能和实现差异，指出高级语言绑定和底层优化实现之间可能存在显著行为差距^[9]。Bos 等人评估了 Kyber 的移植实现并发现早期版本存在数据相关操作，说明即使遵循规范的实现也可能包含侧信道隐患^[10]。这些工作共同表明，实现级独立评估是标准化之外的必要步骤。

我们使用的 `pqcrypto` 绑定属于应用开发者较容易接触到的软件接口。对这类库进行黑盒端到端计时测试，不能替代源码审计或硬件侧信道评估，但可以帮助开发者快速判断常见调用方式下是否出现明显可分辨的时间异常。与仅关注单一参数集的测试相比，全参数集实验能够进一步观察安全级别提升和数据规模增大是否改变时间行为。

2.2 时间侧信道与实现安全

时间侧信道攻击的核心思想是通过观测程序运行时间推断秘密相关信息。Kocher 证明了不同私钥比特引发的模幂运算路径差异可以通过大量计时观测恢复^[4]。后续远程计时研究进一步说明，即使通过网络进行测量，时序差异仍可能在现实环境下被利用^[11]。Bernstein 对 AES 的缓存时序攻击则表明，侧信道并不只来自显式分支，内存访问和缓存命中状态同样可能泄露信息^[5]。

软件时间差异的来源较为复杂。除秘密相关分支外，表查找、缓存未命中、数据相关乘法、异常处理、内存分配、解释器开销和操作系统调度都可能改变单次调用时间。对于密码实现而言，常数时间编程通常要求控制流和内存访问不依赖秘密数据，但在高级语言绑定和通用操作系统环境中，调用边界之外的噪声同样会进入观测结果。因此，端到端软件计时既有部署相关性，也需要通过重复采样、随机化采集顺序和对照实验降低误判风险。

这些研究推动了常数时间编程和实现级安全评估的发展。对于后量子密码算法，虽然其数学困难性不同于 RSA 和椭圆曲线密码，但底层软件仍运行在相同的处理器、缓存和操作系统之上，因此同样需要关注输入相关的执行时间差异。我们将执行时间作为可观侧信道，关注攻击者可控输入是否会在 ML-KEM 解封装实现中诱发稳定的时间差异。

2.3 泄漏检测与统计评估方法

侧信道评估中常见的一类方法是测试向量泄漏评估 (TVLA)。TVLA 通常构造两类输入或中间状态，通过统计检验判断观测轨迹是否存在显著差异^[8]。其优点是不需要完整攻击流程，也不需要预先知道泄漏模型，适合作为实现进入深入审计前的筛查工具。其局限也很明确：统计显著只能说明观测分布不同，并不自动说明差异可以用于密钥恢复；反过来，未通过某一组测试也不能证明实现不存在其他形式的泄漏。

在软件计时场景下，样本分布通常带有长尾和异方差特征。单纯比较均值容易遗漏分布形状变化，单纯依赖 p 值又可能把样本量带来的极小差异误读为实践风险。因此，我们借鉴 TVLA 的两类输入思想，以 Welch's t 检验^[12]、KS 检验和 Cohen's d 效应量^[13] 作为正文中的主要统计证据，并在自动化报告中保留 Mann-Whitney U 检验^[14] 作为非参数辅助诊断。

此外，泄漏检测本身也需要对误报和漏报保持谨慎。若采集顺序固定，系统负载或温度变化可能被误认为标签差异；若同一基础密文派生出的样本同时出现在训练集和测试集，分类器可能记住密文组而不是学习泄漏信号。我们在实验设计中引入随机打乱、分组交叉验证、多轮重复、固定延迟正对照和延迟灵敏度扫描，目的就是让负检测结论具有更清楚的边界。

2.4 格基 KEM 的侧信道研究

CRYSTALS-Kyber/ML-KEM 属于格基 KEM，其解封装过程包含解密、重加密、密文比较和失败路径密钥派生等步骤。Ravi 等人从通用角度研究了 CCA 安全格基 PKE/KEM 的侧信道问题，指出若实现泄露解封装成功与失败路径差异，可能为选择密文攻击提供入口^[6]。Ueno 等人提出“重加密诅咒”，展示 FO 变换中的重加密步骤在功耗与电磁信道下具有泛化的攻击面^[7]。Primas 等人则从单轨迹功耗分析角度说明，即便格密码实现采用一定防护，细粒度实现特征仍可能被分析利用^[15]。

这些工作说明，格基 KEM 的实现安全并不只取决于底层困难问题。特别是在 CCA 安全变换下，解封装需要先根据私钥处理输入密文，再通过重加密或比较步骤决定最终共享密钥的来源；若成功路径与失败路径在时间、功耗或电磁轨迹上可区分，攻击者就可能获得超出标准接口的额外信息。因此，有效密文与无效密文的对照测试是 KEM 实现评估中的自然实验设置。我们使用同长度无效密文，正是为了尽量排除长度错误带来的显式异常路径，把关注点集中在内容差异是否影响正常解封装流程。

与功耗和电磁分析相比，软件计时实验的攻击门槛更低，但信噪比也更差，因此将其定位为早期筛查方法，而非完整攻击方法。检测到信号说明实现值得进一步审计；未检测到信号则只能说明当前条件下未发现可区分差异。

值得特别关注的是，KyberSlash 工作报告了 Kyber/ML-KEM 多个实现（包括参考实现）中确实存在的密钥相关时序泄漏：在 poly_tomsg 等模块化压缩步骤中，对模数 q 的变长除法运算耗时与中间值相关，攻击者可在 Cortex-A7 和 Cortex-M4 等平台上通过该时序差异在数分钟到数小时内恢复密钥^[16]。这一案例表明，ML-KEM 的时间侧信道泄漏并非纯理论假设，而是已在真实实现中被发现并修复的问题。该工作同时提出基于 Valgrind 的指令级污点分析方法，用于在源码层面定位变长时间操作。与之的主要区别在于观测粒度：KyberSlash 类工作针对单条指令或单个算术运算的执行时间，而本研究从输入（有效/无效密文）到端到端解封装耗时这一更粗粒度的黑盒视角出发，因此二者在检测能力和适用阶段上是互补的。这也意味着，即便被测实现在某些指令层面仍存在类似 KyberSlash 的亚微秒级时序差异，该差异也可能被软件计时噪声淹没而不在当前观测粒度下形成稳定信号；因此本研究端到端的负检测结论与 KyberSlash 工作的发现并不矛盾，关于这一粒度边界的进一步讨论见第 6 节。

2.5 机器学习辅助泄漏检测

机器学习方法已广泛用于侧信道分析中的模板攻击和特征识别。Lerman 等人比较了随机森林、支持向量机和 k 近邻等模型在功耗分析中的表现^[17]；Cagli 等人将卷积神经网络用于带有抖动对抗措施的功耗轨迹，展示了深度学习方法对复杂轨迹错位的适应能力^[18]。但深度模型通常需要大量轨迹和更高训练成本，且可解释性较弱。

我们面向小样本、端到端、筛查型的软件计时任务，选择逻辑回归、线性 SVM、随机森林和直方图梯度提升树作为检测模型。这些模型既能覆盖线性与非线性判别，又便于与统计检验和特征重要性分析结合。机器学习分类结果被作为统计检验之外的补充证据，而不是以模型准确率替代传统泄漏评估；模型置换检验仅作为输出报告中的辅助诊断，不进入正文泄漏判定条件。

需要注意的是，机器学习方法用于泄漏检测时尤其容易受到数据划分方式影响。如果训练集和测试集共享同一基础密文组，模型可能学习到某些偶然的组内计时特征，从而给出虚高的测试准确率。另一方面，在没有真实信号的数据上，复杂模型也可能通过过拟合产生局部波动。因此，我们使用分组交叉验证约束同一基础密文组不会同时出现在训练集和测试集，并将模型准确率与统计检验、效应量和正对照结果一起解释。这样做的目标不是追求最高分类性能，而是让机器学习结果成为更稳健的泄漏筛查证据。

3 理论基础

本节先说明 ML-KEM 的参数集和基于 Module-LWE 的安全性论证，再转向解封装流程中与实现侧信道相关的步骤。这样的顺序对应本研究的基本立场：ML-KEM 的规范安全目标有明确数学依据，但部署时仍需独立检查具体实现是否在时间行为上泄露额外信息。

3.1 ML-KEM 参数集

ML-KEM 是一种密钥封装机制，包含密钥生成、封装和解封装三个接口。其安全性基于模格误差学习问题，底层多项式环通常记为 $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ ，其中 $q = 3329$ 。三个参数集的主要差异在于模块维度 k 和压缩参数，进而影响公钥、密文、私钥大小和计算开销。表 1 列出 FIPS 203 中与本研究实验相关的主要参数^[2]。

表 1 ML-KEM 三个参数集的接口规模

参数集	NIST 安全级别	公钥大小 (B)	密文大小 (B)	私钥大小 (B)
ML-KEM-512	1	800	768	1632
ML-KEM-768	3	1184	1088	2400
ML-KEM-1024	5	1568	1568	3168

3.2 ML-KEM 的数学原理与抗量子安全性论证

ML-KEM 可以理解为在多项式环上构造的带噪声线性代数系统。令

$$R_q = \mathbb{Z}_q[X]/(X^n + 1), \quad n = 256, \quad q = 3329, \quad (1)$$

其中 R_q 中的元素是次数小于 256 的多项式，系数按模 q 计算，并满足 $X^{256} \equiv -1$ 。ML-KEM 的向量和矩阵不是定义在普通整数域上，而是定义在 R_q 上：公开矩阵 $A \in R_q^{k \times k}$ ，秘密向量 $s \in R_q^k$ ，误差向量 $e \in R_q^k$ 。密钥生成阶段的核心关系可抽象为

$$t = As + e \pmod{q}. \quad (2)$$

其中 A 和 t 构成公钥的主要部分， s 是私钥的主要秘密。若没有噪声项 e ，式 (2) 只是普通线性方程组，攻击者可以用线性代数方法求解 s ；而加入小噪声后，攻击者看到的是“近似正确”的线性关系，求解 s 变成 Module-LWE 问题。

噪声的“小”和模数的“大”共同保证正确性与安全性的平衡。封装时，发送方采样短向量 r, e_1 和短多项式 e_2 ，并将消息 m 编码到模 q 的高低两个区域中。忽略压缩和哈希细节后，底层公钥加密可写成

$$u = A^T r + e_1 \pmod{q}, \quad (3)$$

$$v = t^T r + e_2 + \text{encode}(m) \pmod{q}. \quad (4)$$

持有私钥的接收方计算

$$v - s^T u = \text{encode}(m) + e^T r + e_2 - s^T e_1 \pmod{q}. \quad (5)$$

式 (5) 中剩余的 $e^T r + e_2 - s^T e_1$ 是解密噪声。ML-KEM 通过参数选择和压缩设计，使该噪声以极高概率小于判决边界，因此合法密文可以恢复 m ；但从攻击者角度看，正是这些小噪声隐藏了 s ，使公钥样本难以与随机样本区分。三个参数集的 k 分别为 2, 3, 4， k 越大，模块维度越高，攻击者需要处理的格维数和计算成本也随之增大。

从安全性论证角度看，ML-KEM 依赖的是 Module-LWE，而不是 RSA 的大整数分解或椭圆曲线密码的离散对数。LWE 问题具有从最坏情况格问题到平均情况样本的安全归约^[19]；Module-LWE 是在带有代数结构的模块格上折中安全性与效率的变体，也是 Kyber/ML-KEM 得以获得较小密钥和较高速度的原因之一^[3,10]。这里的安全性不是经验猜测，而是以可归约的困难问题为基础：若存在一个多项式时间攻击者能够从 (A, t) 中恢复 s ，或能够稳定区分 $(A, As + e)$ 与均匀随机样本，则可构造一个算法来求解或区分相应的 Module-LWE 实例。这与直接解线性方程不同；攻击者面对的是大量带小噪声的模环线性关系，而噪声项正是安全归约中的核心屏障。

更具体地，底层公钥加密的机密性可以通过一系列混合实验描述。首先，将公钥中的 $t = As + e$ 替换为均匀随机向量；如果攻击者能察觉这一替换，就得到一个 Module-LWE 区分器。其次，将密文中的 $u = A^T r + e_1$ 和 $v = t^T r + e_2 + \text{encode}(m)$ 逐步替换为随机量；若攻击者仍能区分消息 m ，同样可转化为对 Module-LWE 样本与随机样本的区分。因此，在 Module-LWE 假设成立时，底层加密对被动窃听攻击具有语义安全性。ML-KEM 作为 KEM 还需要抵抗选择密文攻击，因此规范采用 Fujisaki-Okamoto 型变换：解封装时重新封装候选消息并以常数时间比较密文，验证失败时输出由私钥随机种子和密文派生的隐式拒绝密钥。该变换把底层加密的机密性提升为 KEM 接口下的 IND-CCA 型安全性，这也是算法 1 中重加密验证步骤存在的原因^[3,10]。

参数层面，ML-KEM-512、ML-KEM-768 和 ML-KEM-1024 分别对应 NIST 安全级别 1、3 和 5^[2]。这些级别并不是简单按密钥长度命名，而是根据公开已知的经典和量子攻击成本进行估计：攻击者通常需把 Module-LWE 实例转化为模块格上的短向量问题或近似最近向量问题，再使用格约化、枚举、筛法及其混合算法求解。参数集的模块维度 $k = 2, 3, 4$ 逐步增大，压缩和噪声分布也随之调整，使已知攻击的成本分别达到对应安全级别所要求的量级。换言之，ML-KEM 的安全性来自“困难问题归约 + 具体参数攻击成本评估”的组合，而不是单纯依靠实现复杂或攻击者不了解算法。

抗量子安全性也可以从这一点得到解释。Shor 算法能够在量子计算机上多项式时间求解分解和离散对数问题^[1]，因此直接威胁 RSA、Diffie-Hellman 和椭圆曲线密码；但目前没有已知量子算法能以类似方式在多项式时

间内解决 LWE 或 Module-LWE。量子计算可以带来若干加速，例如 Grover 算法对无结构搜索给出平方级加速^[20]，部分量子筛法也可能改善格攻击中的常数或指数因子；这些影响已被纳入后量子参数选择和安全级别评估中。因此，严格说 ML-KEM 不是被证明“永远安全”，而是在标准计算安全意义下：只要 Module-LWE 对已知经典和量子多项式时间算法仍然困难，且参数攻击成本达到对应级别，ML-KEM 就被认为满足相应的后量子安全目标。

需要强调的是，数学安全性与实现安全性是两个层次。式 (2) 到式 (5) 说明的是规范算法为何具有基于 Module-LWE 的计算安全性；后续时间侧信道分析则关注具体程序在执行这些运算时是否暴露额外信息。即使底层 Module-LWE 问题仍然困难，若实现中的分支、内存访问或算术指令耗时依赖秘密数据，攻击者仍可能绕过数学困难性，从侧信道获得密钥相关信息。

3.3 ML-KEM 解封装流程

本研究重点测量解封装阶段。如算法 1 所示，ML-KEM 解封装的核心步骤如下：首先用私钥解密密文得到候选消息 m' ；然后以 m' 为输入重新运行封装过程，得到候选共享密钥 K' 和重构密文 c' ；最后以常数时间比较 c' 与输入密文 c 是否相同，若相同则输出 K' ，否则输出由私钥随机种子和 c 派生的隐式拒绝密钥 $\bar{K}$ 。这一结构来自 Fujisaki-Okamoto (FO) 变换，是 ML-KEM 实现 CCA 安全性的关键机制^[3]。

算法 1 ML-KEM 解封装核心流程（简化）

输入：私钥 dk ，密文 c

输出：共享密钥 K

```

1:  $m' \leftarrow \text{decrypt}(dk, c)$                                 ▷ 用私钥解密
2:  $(K', c') \leftarrow \text{encaps}(pk, m')$                         ▷ 重新封装
3: if  $c' = c$  （常数时间比较） then                        ▷ FO 验证
4:   return  $K'$                                               ▷ 有效路径
5: else
6:   return  $\bar{K} \leftarrow \text{prf}(\text{seed}, c)$                       ▷ 隐式拒绝
7: end if

```

若实现未能保证各步骤严格常数时间执行，则有效密文（验证通过）与无效密文（验证失败）可能在解密、重加密或比较步骤中产生可观测的时间差异。我们将解封装端到端时间作为可观测信道，通过对比有效密文与四类无效密文，检测是否存在此类稳定差异。

3.4 统计检验与效应量

设有效密文样本的聚合均值时间为 $\{x_i\}_{i=1}^{n_0}$ ，无效密文样本为 $\{y_i\}_{i=1}^{n_1}$ 。Welch's t 检验用于判断两组均值是否存在显著差异，且不要求两组方差相等^[12]：

$$t_W = \frac{\bar{x} - \bar{y}}{\sqrt{s_x^2/n_0 + s_y^2/n_1}}. \quad (6)$$

Welch's t 检验主要用于判断两类输入的平均解封装时间是否存在显著差异。Mann-Whitney U 检验不依赖正态分布假设，更适合长尾计时数据^[14]。KS 检验比较两组经验分布函数的最大距离，可检测均值之外的整体分布形状变化。Cohen's d 用于衡量差异的实践大小^[13]：

$$d = \frac{\bar{y} - \bar{x}}{\sqrt{(s_x^2 + s_y^2)/2}}. \quad (7)$$

其中，KS 检验用于判断两类时间分布整体形态是否不同；Cohen's d 用于判断差异是否具有实际效应，而不只是统计显著。我们将 $p < 0.01$ 作为主要统计显著性阈值，将 $|d| \geq 0.2$ 作为可解释效应的最低阈值。

3.5 分组机器学习评估

在分类检测中，特征向量由每条聚合轨迹的 13 维统计量构成，标签表示有效密文或无效密文。我们使用平衡准确率作为主要指标：

$$\text{balacc} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right). \quad (8)$$

平衡准确率用于衡量模型对有效密文和无效密文的泛化区分能力。两类样本数量相等时，随机猜测的期望平衡准确率为 0.5。为避免模型记住同一基础密文组的偶然计时特征，我们使用 `GroupShuffleSplit`，要求同一 `group_id` 下的有效样本和无效样本始终同时位于训练集或测试集一侧。随机森林使用 Breiman 的集成树模型^[21]，梯度提升树参考提升树的逐步残差拟合思想^[22]，模型实现基于 `scikit-learn`^[23]。

4 实验设计与实现

4.1 总体技术路线

检测流程由输入构造、计时采集、特征提取、统计检验、机器学习评估和泄漏判定六个部分组成。整体架构如图 1 所示：首先针对三个 ML-KEM 参数集生成有效密文，并按照四种策略构造同长度无效密文；随后在真实场景和固定延迟正对照场景下采集解封装时间；最后将原始计时聚合为统计特征，并用多种统计检验和分类模型交叉判断是否存在稳定可区分的时间差异。



图 1 ML-KEM 解封装时间侧信道泄漏检测整体架构

4.2 威胁模型与观测模型

本研究考虑的攻击者是与被测程序运行在同一台机器上的本地、协同驻留进程。该攻击者具备类似 CCA 模型中的选择密文能力：可以构造任意同长度密文并提交给目标实现的解封装接口，但不能访问私钥或其内部状态。与标准 CCA 安全模型不同的是，该攻击者额外具备本地高精度时间侧信道观测能力；其通过被测进程自身提供的高精度计时接口（Python `time.perf_counter_ns`）获取每次解封装调用的墙钟时间，不依赖缓存探测、性能计数器或功耗/电磁探针等更强的旁路观测手段。

观测模型上，我们不直接使用单次调用的原始计时，而是将连续 $R = 50$ 次解封装的耗时聚合为均值等 13 维统计特征，再据此判断有效密文与无效密文是否可区分。这一设置对应攻击者可以多次重放同一密文并进行统计平均的场景，但不对应一次性观测、网络传输延迟主导或攻击者只能获得粗粒度计时（如毫秒级）的更弱场景——在后两种场景下，所提方法提供的灵敏度上界会进一步劣化。判定结论应在该观测模型下理解：若在本机、可重复采样、纳秒级精度的有利条件下仍未检测到稳定差异，则在更弱的观测条件（如 Brumley 和 Boneh 讨论的远程网络计时^[11]）下检测到差异的可能性只会更低，而不会更高。

4.3 实验对象与运行环境

主实验对象为 Python pqcrypto==0.3.4 包提供的 ML-KEM 三个参数集绑定（512、768 和 1024），运行于 Windows 11 x86_64 平台（表 2）。补充实验在 macOS arm64（Apple Silicon，macOS 26.5）上另以 pqcrypto==0.3.4 和 liboqs（Python 绑定）分别作为被测后端，两种后端均覆盖全部三个参数集，Python 版本为 3.9.6，分析依赖版本与主实验一致。

表 2 主实验运行环境（Windows 11 x86_64）

项目	配置
CPU	Intel Core i7-12700K（支持 AVX2）
操作系统	Windows 11 x86_64（10.0.26200）
Python	3.12.12（Anaconda 发行版）
ML-KEM 实现	ml_kem_512/768/1024
pqcrypto	0.3.4
numpy	2.0.2
scipy	1.13.1
scikit-learn	1.6.1
matplotlib	3.9.4

4.4 无效密文策略

每个基础密文组首先通过封装得到一个有效密文，然后构造同长度的无效密文。我们使用四种策略，见表 3。这四类策略覆盖了从接近合法密文到完全非结构化输入的不同扰动强度：single_bit 和 byte_flip 用于观察局部扰动是否触发解封装路径差异；random_bytes 用于模拟一般随机无效密文；zero 则用于测试极端结构化输入是否造成异常时间行为。

表 3 无效密文构造策略

策略	构造方式	目的
single_bit	基于 SHA-256 派生字节位置和比特位，仅翻转一位	最小扰动输入
byte_flip	基于 SHA-256 派生字节位置，并将该字节异或 0xFF	中等局部扰动
random_bytes	生成同长度确定性伪随机字节串	模拟随机无效密文
zero	生成同长度全零密文	模拟结构化极端输入

所有策略均保持密文长度不变，以避免将长度错误引起的异常处理路径混入主要实验信号。对于 single_bit 策略，扰动位置由

$$h = \text{sha256}(\text{le32}(g)), \quad \text{idx} = \text{int}(h_0, \dots, h_3) \bmod |\text{ct}|, \quad \text{bit} = h_4 \bmod 8$$

(9)

确定，其中 g 为基础密文组编号。

4.5 采集流程

每次实验首先生成一个 ML-KEM 密钥对，并生成 60 个基础有效密文组。正式采集前执行 500 次解封装预热，预热过程在有效密文和无效密文之间交替，以减少某一类样本获得更热缓存状态的系统偏差。随后为每类构造 400 条聚合采集任务，随机打乱后逐条执行。每条聚合轨迹重复执行 50 次解封装，并记录纳秒级原始时间。需要说明的是，400 条聚合样本来自 60 个基础密文组，因此逐轨迹统计检验用于描述端到端计时分布差异；涉及跨基础密文泛化的机器学习评估和稳健性判断则按 group_id 分组处理，并在数据质量报告中额外保留组级配对检验。整体流程如算法 2 所示。

算法 2 ML-KEM 解封装时间采集流程

输入：参数集 v ，无效密文策略 s ，组数 $G = 60$ ，每类样本数 $N = 400$ ，重复次数 $R = 50$

输出：真实场景与正对照场景下的特征表和分析报告

```

1:  $(pk, sk) \leftarrow \text{keygen}_v()$ 
2: for  $g = 0$  to  $G - 1$  do
3:    $(ct_g, K_g) \leftarrow \text{encaps}_v(pk)$ 
4:    $\tilde{ct}_g \leftarrow \text{invalid}(ct_g, g, s)$ 
5: end for
6: 交替执行 500 次  $\text{decaps}_v(sk, ct_g)$  与  $\text{decaps}_v(sk, \tilde{ct}_g)$ 
7: 构造有效/无效两类采集任务并随机打乱
8: for 每个采集任务  $(g, label, ct)$  do
9:   重复执行  $R$  次解封装计时
10:  从 50 个原始时间提取 13 维统计特征
11: end for
12: 对真实场景和 20,000 ns 正对照分别执行统计检验与模型评估
  
```

4.6 特征与判定规则

对每条聚合轨迹提取的 13 维特征包括：均值、中位数、标准差、最小值、最大值、第 10 百分位数、第 90 百分位数、四分位距、中位绝对偏差、5% 截尾均值、偏度、超额峰度和变异系数。线性模型使用 `RobustScaler` 做稳健缩放并裁剪极端值；树模型直接使用原始特征。

软件计时数据容易受到操作系统调度、缓存状态、动态频率和后台进程影响，单一统计量可能出现偶然显著。因此，我们不将某一个 p 值或某一次分类准确率作为泄漏证据，而采用分类可分性、统计显著性和效应量共同满足的联合判定规则。该设计用于降低误报风险，使泄漏判定更偏向保守。

真实场景仅当以下条件全部成立时报告检测到泄漏：最佳模型平衡准确率不低于 0.65，Welch's t 检验 $p < 0.01$ ，且 $|d| \geq 0.2$ 。正对照要求最佳模型平衡准确率不低于 0.90，且 Welch's t 检验 $p < 0.001$ 。这一区分保证了真实场景判断不会只依赖单一统计量，同时正对照用于确认管线确实能识别已知时间信号。

4.7 数据集构成与开放获取

实验数据为自行采集的 ML-KEM 解封装时间测量数据，不依赖任何外部数据集。

数据规模。主实验 (Windows 11 x86_64, `pqcrypto==0.3.4`) 共执行 $5 \times 3 \times 4 = 60$ 次完整实验，每次实验在真实场景和正对照场景下各采集两类标签 (有效密文、无效密文) 各 400 条聚合样本，共 $60 \times 2 \times 2 \times 400 = 96,000$ 条。macOS arm64 补充实验另有 $2 \times 3 \times 3 \times 4 \times 2 \times 400 = 172,800$ 条 (`pqcrypto` 和 `liboqs` 两后端各 3 轮)。全部数据均由项目采集脚本自动生成，无人工标注。

样本结构。每条聚合样本由 50 次重复解封装的原始纳秒级时间戳提取 13 维统计特征构成 (均值、中位数、标准差、最小值、最大值、 p_{10} 、 p_{90} 、四分位距、中位绝对偏差、截尾均值、偏度、超额峰度、变异系数)，标签为 0 (有效密文) 或 1 (无效密文)，并附带参数集、无效密文策略、基础密文组编号 (`group_id`) 等元数据字段。

文件格式。每次实验输出目录包含：原始纳秒时间序列 (`real_raw_timings.csv`、`positive_control_raw_timings.csv`)、13 维聚合特征矩阵 (`real_traces.csv`、`positive_control_traces.csv`)、统计与机器学习分析报告 (`real_analysis.json`、`summary.json`) 和可视化图表。

数据获取。完整原始数据已随论文开放发布，可从项目仓库 Release 页面下载：<https://github.com/AndrewAccuracy/PostQuantum/releases>。仓库同时提供采集脚本，允许在安装相同版本依赖后独立复现全部数据。

5 实验结果

5.1 数据完整性与总体概况

正式实验覆盖 5 轮独立重复、3 个参数集和 4 种无效密文策略，共 $5 \times 3 \times 4 = 60$ 次完整实验。每次实验包含真实场景和正对照场景，每个场景两类标签各 400 条聚合样本，因此共有 96,000 条聚合样本；每条样本包

含 50 次重复计时，对应约 480 万次原始解封装计时。数据质量报告显示：60 次实验均完成，标签平衡、缺失值、非有限数值和重复 `trace_id` 检查均通过；所有正对照管线均有效。

跨 60 次真实场景实验，扰动密文相对有效密文的平均时间差均值为 81.6 ns，标准差为 1044.9 ns，最小值和最大值分别为 -1691.5 ns 与 2014.8 ns。最佳真实模型平衡准确率均值为 0.5134，范围为 0.4761–0.5566；正对照最佳模型平衡准确率均值为 0.9995。

5.2 时间分布与统计检验

图 2 给出所有参数集与策略汇总后的真实场景和正对照时间分布。真实场景中，两类样本的主体分布高度重叠，未出现稳定可见的分离；正对照中，无效密文类因额外 20,000 ns 忙等待延迟整体右移，分布差异清晰，说明计时和分析流程能够捕获已知信号。这一视觉重叠在量级上与统计指标高度吻合：跨 60 次实验的平均时间差仅 81.6 ns，而组内标准差约为 1044.9 ns，信噪比不足 0.08，即分布中心差异远小于任意一类样本自身的波动范围。

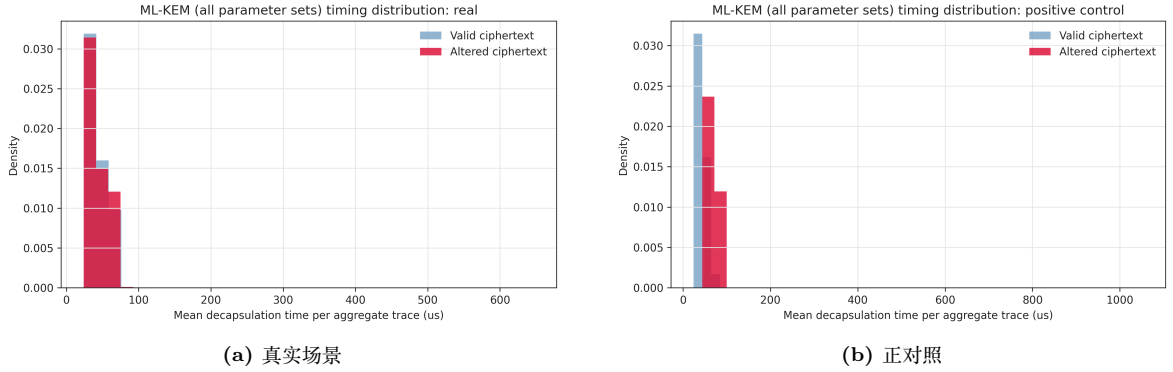


图 2 所有参数集与无效密文策略汇总后的解封装平均时间分布。

进一步地，图 3 展示了 `single_bit` 策略下三个参数集在 5 轮真实实验中的时间分布。随着参数集从 512 增大到 1024，平均解封装时间整体上升（5 轮均值分别约为 28.25、41.96 和 60.12 μs ），但计算量增大并未带来可分辨的标签差异：三个参数集内部的有效/无效密文分布均保持高度重叠，且组内波动随参数集增大同步增长，未出现某一参数集相对噪声显著降低、进而使标签分离更易检测的情况。

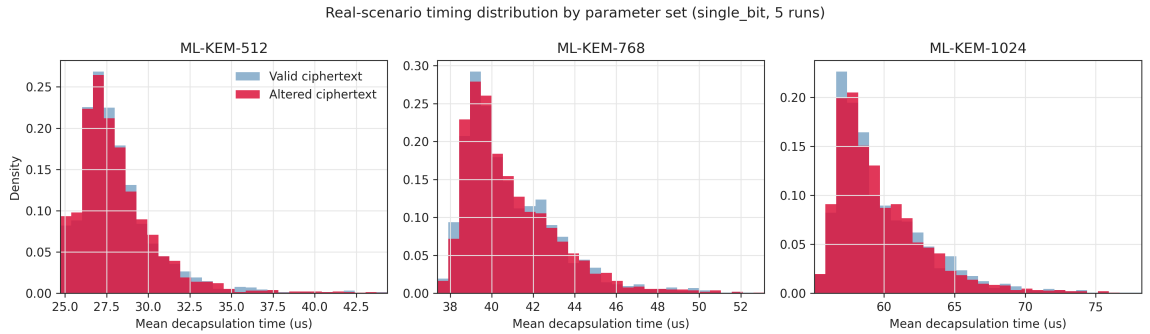


图 3 `single_bit` 策略下三个参数集的真实场景时间分布（5 轮合并）。

以上视觉观察在统计层面得到精确量化。表 4 汇总了 12 个参数集/策略组合在 5 轮真实实验中的均值结果。所有组合均未触发泄漏判定；KS 检验均值未形成稳定显著模式；Cohen's d 的组合均值绝对值最高仅为 0.0448，低于可忽略效应阈值 0.2。为避免负结果表格失去阅读焦点，表中用浅色阴影标出每一种无效密文策略下真实最佳平衡准确率最高的参数集。即便这些阴影行也仍远低于 0.65 的泄漏判定阈值，说明它们只是相对更接近随机波动上沿，而不是可疑阳性结果。表中 Welch p 列仅作为 5 轮结果的描述性摘要，不作为新的假设检验。

为避免把 p 值均值误读为显著性证据，我们同时检查了 60 次单次实验的 Welch's t 检验结果：最小单次 p 值为 0.0341，仅 2 次低于 0.05，且没有一次低于 0.01。若按 60 次比较做 Bonferroni 校正，族系显著性水平 0.01 下的单次阈值为 $0.01/60 \approx 1.67 \times 10^{-4}$ ，所有结果均远未达到该阈值。进一步地，数据质量报告中的组级配对检验最小 p 值为 0.0377，也没有低于 0.01。这些结果说明，逐轨迹检验和组级检验均未形成可支撑泄漏判

定的稳定统计证据。值得注意的是，四种无效密文策略——从最小扰动 (single_bit)、局部破坏 (byte_flip) 到随机无效输入 (random_bytes) 和结构化极端输入 (zero) ——均给出了一致的负结论，说明结果不依赖于特定的密文扰动方式，而是在从最小改动到最大结构差异的整个输入范围内都成立。

表 4 12 个参数集/策略组合的真实场景与正对照结果 (5 轮均值)

参数集	策略	$\Delta\mu$ ns	Welch p	KS p	Cohen's d	真实最佳准确率	正对照最佳准确率
512	single_bit	-31.2	0.275	0.438	-0.0031	0.515	1.000
512	byte_flip	-83.1	0.284	0.383	0.0253	0.517	1.000
512	random_bytes	-210.8	0.428	0.475	-0.0075	0.506	1.000
512	zero	988.6	0.327	0.561	0.0447	0.528	1.000
768	single_bit	-780.4	0.284	0.695	-0.0448	0.505	1.000
768	byte_flip	636.8	0.416	0.436	0.0224	0.510	1.000
768	random_bytes	177.6	0.608	0.531	-0.0020	0.511	1.000
768	zero	-225.9	0.333	0.722	-0.0118	0.505	0.999
1024	single_bit	6.1	0.441	0.372	-0.0022	0.522	0.998
1024	byte_flip	720.2	0.578	0.516	0.0350	0.506	0.999
1024	random_bytes	397.0	0.357	0.558	0.0076	0.513	1.000
1024	zero	-616.3	0.262	0.493	-0.0442	0.523	0.999

阴影行和加粗值表示同一无效密文策略下真实最佳平衡准确率最高的参数集；这些值仍明显低于 0.65 判定阈值。

5.3 机器学习分类结果

表 5 汇总了 60 次实验中四种分类器的平均表现。在标签均衡的二分类任务中，平衡准确率 0.5 对应随机猜测水平。如果模型在分组交叉验证下长期接近 0.5，说明其无法在未见过的基础密文组上学习到稳定的有效/无效密文区分边界。真实场景下所有模型的平衡准确率均接近 0.5，正对照中四种模型则接近完美分类，进一步验证检测管线有效。

表 5 四种模型跨 60 次实验的平均平衡准确率

模型	真实场景均值	真实场景标准差	正对照均值	正对照标准差
逻辑回归	0.4996	0.0190	0.9987	0.0016
线性 SVM	0.4986	0.0152	0.9989	0.0014
随机森林	0.4976	0.0198	0.9993	0.0011
梯度提升树	0.4961	0.0188	0.9989	0.0017

为进一步检查模型分数是否只是训练/测试划分的偶然波动，我们对代表性运行(ML-KEM-768/single_bit)执行基于分组交叉验证的置换检验。检验时保持特征矩阵和分组结构不变，仅随机打乱标签，重复计算模型平衡准确率以形成零假设分布。如图 4 所示，四种模型的观测分数均落在置换零分布中心区域附近，置换 p 值均明显高于 0.05，说明真实场景下模型近 0.5 分数与随机标签情形不可区分，未学习到可泛化的有效/无效密文判别模式。

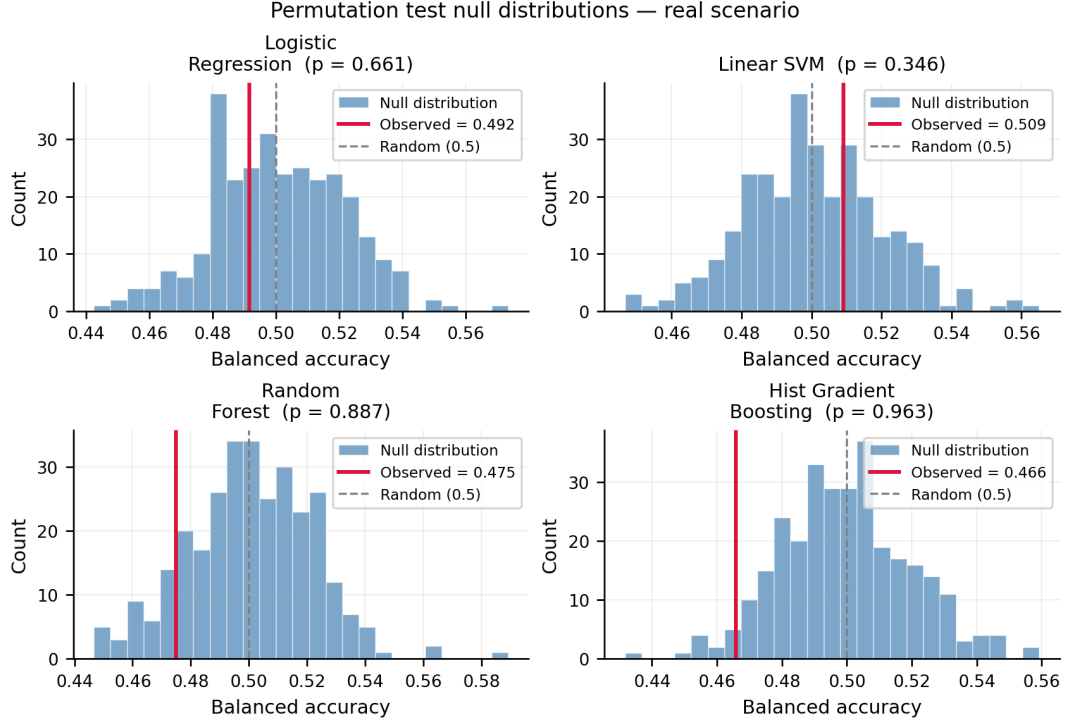


图 4 代表性运行 (ML-KEM-768 / `single_bit`) 中四种模型的置换检验零分布 (真实场景)。红色竖线为观测平衡准确率, 灰色虚线为随机基线 0.5。

图 5 从参数集和无效密文策略两个维度展示最佳模型平衡准确率。真实场景柱形图始终贴近随机基线 0.5, 而正对照均接近 1.0。这表明不同安全级别和不同无效密文构造方式没有改变总体结论。

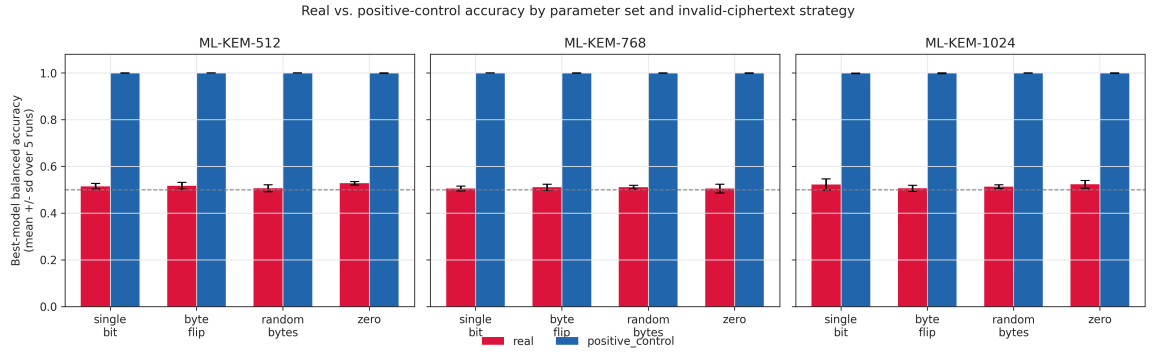


图 5 各参数集和无效密文策略下真实场景与正对照的最佳模型平衡准确率。

5.4 延迟灵敏度扫描

为理解检测管线的灵敏度, 我们对 ML-KEM-768 与 `single_bit` 策略执行额外的正对照延迟扫描, 延迟档位为 500、1,000、2,000、5,000、10,000 和 20,000 ns。结果见表 6 和图 6。500 ns 时, KS 检验已经检测到分布形态变化, 但均值检验和效应量尚未达到泄漏判定阈值; 5,000 ns 时, 最佳模型准确率、Welch 检验和效应量首次同时满足真实场景三条件判定规则; 10,000 ns 以后, 正对照信号非常稳定。

表 6 ML-KEM-768 / `single_bit` 延迟灵敏度扫描

延迟 (ns)	最佳平衡准确率	Welch p	KS p	Cohen's d
500	0.633	1.16×10^{-1}	3.25×10^{-4}	0.112
1000	0.634	7.31×10^{-1}	6.92×10^{-5}	-0.024
2000	0.738	6.09×10^{-2}	2.63×10^{-10}	0.133
5000	0.885	1.94×10^{-45}	1.88×10^{-61}	1.067
10000	0.985	2.20×10^{-97}	5.46×10^{-141}	1.716
20000	0.998	$< 10^{-300}$	3.40×10^{-234}	5.548

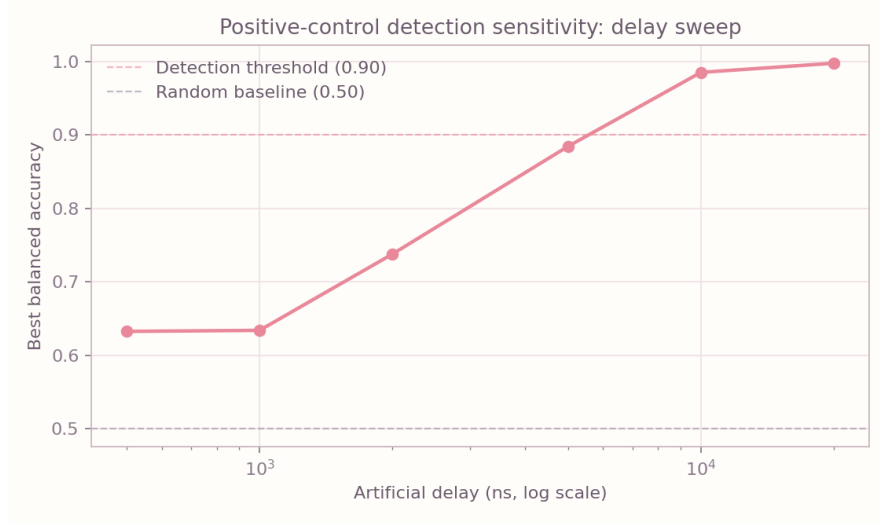


图 6 正对照延迟扫描中最佳模型平衡准确率随人为延迟的变化。

为将上述经验观察转化为可量化的灵敏度参考，我们基于两样本 t 检验的功效近似公式估计逐轨迹样本层面的最小可检测效应量^[13]：

$$d_{\min} \approx (z_{\alpha/2} + z_{\beta})\sqrt{2/n}. \quad (10)$$

在 $\alpha = 0.01$ （双侧， $z_{\alpha/2} \approx 2.576$ ）、每类 $n = 400$ 的设置下，若取检验功效 $1 - \beta = 0.80$ （ $z_{\beta} \approx 0.842$ ），则 $d_{\min} \approx 0.242$ ；判定规则采用的效应量阈值 $|d| \geq 0.2$ 在该样本量下对应的功效约为 60%，略低于 80% 这一常用标准。

将 $d_{\min}$ 换算为绝对时间需要每类聚合轨迹均值（`mean_ns`）的合并标准差。我们对 60 次真实场景实验的原始数据按参数集计算合并标准差，并据此给出两个效应量阈值对应的最小可检测时间差异，结果见表 7。

表 7 各参数集的合并标准差与最小可检测效应（MDE）

参数集	合并标准差 (μs)	MDE @ $ d = 0.2$ (μs , 约 60% 功效)	MDE @ 80% 功效 (μs , $d \approx 0.242$)
ML-KEM-512	9.9	2.0	2.4
ML-KEM-768	14.3	2.9	3.5
ML-KEM-1024	12.8	2.6	3.1

表 7 中 80% 功效对应的逐轨迹最小可检测效应（2.4–3.5 μs ，视参数集而定）与表 6 的延迟扫描结果吻合：2,000 ns 的人为延迟（ $d = 0.133$ ）低于该下界，三条件判定规则未被触发；5,000 ns（ $d = 1.067$ ）已远高于下界，规则首次被全部满足。图 7 在延迟扫描曲线上叠加了 ML-KEM-768 的 MDE 区间（对应表 7 中 $|d| = 0.2$ 到 $d \approx 0.242$ 的 2.9–3.5 μs ），可以看到该区间正好落在曲线由“未达检测阈值”向“稳定检测”过渡的拐点附近，与理论估计一致。

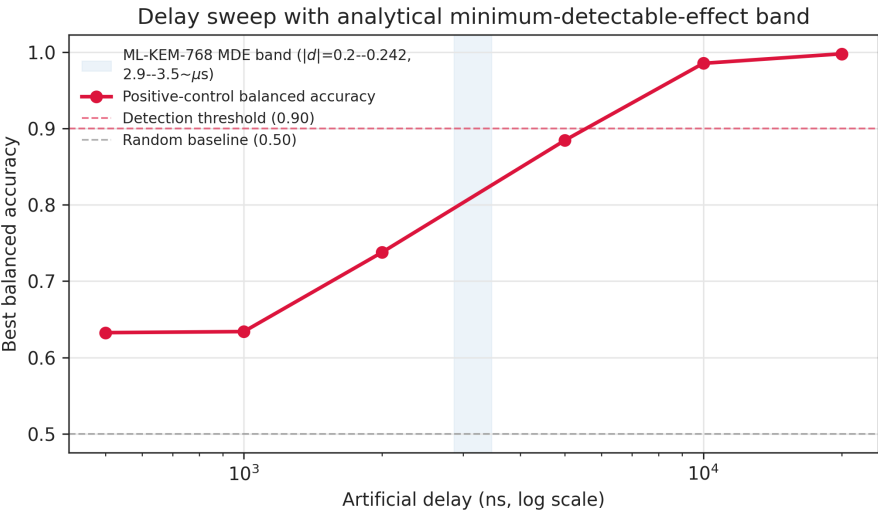


图 7 延迟扫描曲线与 ML-KEM-768 最小可检测效应（MDE）区间的叠加。

因此，本研究真实场景”未检测到泄漏”的结论可更精确地表述为：在逐轨迹近似独立、每类 $n = 400$ 、 $\alpha = 0.01$ 的假设下，当前实验对大约 2.4–3.5 μs （视参数集而定）的稳定均值差异具有约 80% 检出功效。由于这些聚合样本来自 60 个基础密文组，且我们未执行严格的等效性检验，上述功效分析不能被解释为对该量级以下或以上差异的形式化排除；量级更小、更依赖组结构或只出现在特定输入子空间的潜在差异仍不在当前方法的可分辨范围内。

5.5 特征重要性与方法可信度诊断

表 8 展示 `single_bit` 策略下随机森林特征重要性在三个参数集之间的比较。表中按三组平均重要性降序排列，浅蓝行表示总体前三项，粗体表示对应参数集内排名前三的特征。可以看到，较高的重要性主要分布在偏度、峰度、分位数和方差类统计量之间，没有出现单一特征在真实场景中稳定主导分类的现象。这与模型准确率接近随机的结果一致：模型并未找到可泛化的泄漏方向。

表 8 `single_bit` 策略下三个参数集的随机森林特征重要性（5 轮均值）。

特征	ML-KEM-512	ML-KEM-768	ML-KEM-1024	三组平均
skewness	0.092	0.092	0.088	0.091
kurtosis	0.091	0.092	0.086	0.090
p90	0.088	0.091	0.088	0.089
CV	0.097	0.087	0.083	0.089
std	0.097	0.085	0.081	0.088
mean	0.089	0.086	0.081	0.085
max	0.087	0.084	0.080	0.084
trimmed mean	0.085	0.081	0.081	0.082
IQR	0.062	0.074	0.082	0.073
p10	0.060	0.059	0.065	0.061
median	0.055	0.058	0.068	0.060
min	0.058	0.063	0.059	0.060
MAD	0.039	0.048	0.058	0.048

这一均匀分布本身具有独立的诊断意义。若被测实现存在稳定的时序泄漏信号，分类器通常会学习到的区分能力集中于少数特征——例如若均值存在固定偏移，`mean` 的重要性应显著高于其他项；若某分位数反映泄漏分布的尾部特征，对应分位数项应在跨参数集实验中稳定突出。当前结果中 13 个特征的重要性高度分散（最高项偏度三组均值约 0.091，最低项 MAD 约 0.048，极差不足 0.05），且排名靠前的特征在不同参数集之间并不完全一致，说明模型在各参数集实验中倾向于从不同方向拟合随机噪声，而非从某一固定通道提取一致信号。这是机器学习层面对”无结构化泄漏方向”的进一步确认。

图 8 从纵向视角排除了实验间系统性漂移作为负结论替代解释的可能。若实验序列中存在渐进的系统状态变化（如散热累积、后台服务负载或动态频率调整）并与标签产生关联，真实场景的时间差应呈现跨轮次的单调趋势或集中于某一时段；实际结果中差异的符号在 60 次实验间无规律交替，正向与负向差异无明显时序结构，而正对照则始终稳定在接近 20,000 ns 的水平，说明实验间的系统状态变化未对真实场景的标签差异产生可见的方向性干扰。

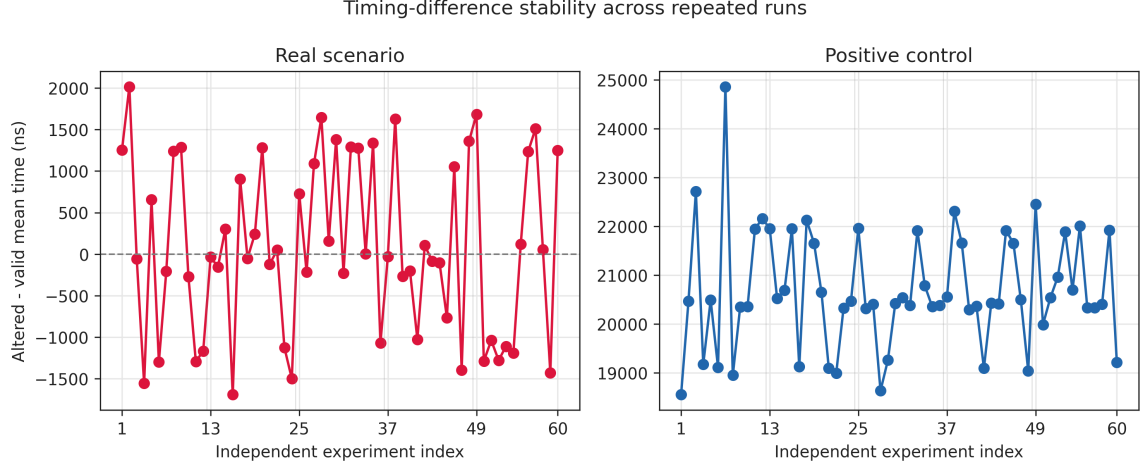


图 8 60 次实验中真实场景与正对照场景的平均时间差稳定性。

图 9 则从实验内部视角，排除了采集顺序与标签分配发生混杂的可能性。该图以代表性运行（ML-KEM-768/single_bit）为例，将采集顺序划分为 20 个相邻区间，在每个区间内计算扰动密文相对有效密文的 10% 截尾均值差。若随机打乱不充分，或系统状态随采集时间漂移并与标签分配产生伪相关，图中应出现单调或周期性的顺序依赖趋势；实际各区间差异围绕 0 波动，误差线频繁覆盖 0，且无明显方向性规律。图 9 与图 8 共同构成双层混淆排查：前者排除实验内部采集顺序的伪相关，后者排除跨实验时序漂移的系统偏差；两个维度的诊断均支持负检测结论不能被方法层面的偏差所解释。

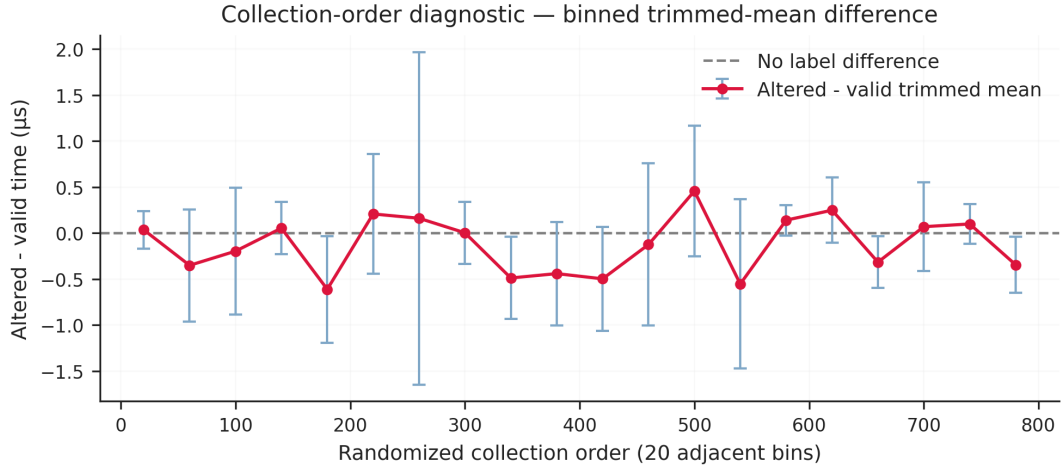


图 9 代表性运行（ML-KEM-768 / single_bit）中真实场景按随机化采集顺序分箱后的标签截尾均值差。点为每个顺序区间内的扰动密文 10% 截尾均值减有效密文 10% 截尾均值，误差线为近似 95% 置信区间。

5.6 跨平台与跨实现补充验证

为验证检测框架在不同处理器架构和不同 ML-KEM 实现上的可复现性，我们还在 macOS arm64（Apple Silicon）平台上使用与主实验完全相同的流水线——每类 400 个样本、50 次重复采集、60 个基础密文组、13 维特征、四种分类器——分别对 pqcrypto 和 liboqs 两种后端各进行 3 轮实验，覆盖全部 3 个参数集和 4 种无效密文策略，每后端共 36 次完整实验（见表 9）。

这一部分不与 Windows 主实验合并估计总体均值,也不作为扩大样本量后的重新检验。原因是平台、Python 构建、动态库实现和底层汇编路径都会同时改变绝对运行时间和噪声结构,直接合并会掩盖实现级差异。我们将其定位为外部一致性检查:若主实验的负结果只是某一平台或某一绑定的偶然现象,那么在相同输入模型和相同判定规则下,补充实验更可能出现相反的统计、效应量或分类证据;反之,若补充实验仍呈现接近随机的分类表现,则说明本研究结论至少没有被这两个新增环境直接推翻。

两种后端在 macOS arm64 上的结果与主实验 (Windows x86_64) 高度一致。pqcrypto 后端 36 次实验均未触发泄漏判定,均值最佳平衡准确率 0.5207,正对照 36/36 通过;liboqs 后端 36 次同样均未触发泄漏判定,均值 0.5349,正对照 35/36 通过 (第 1 轮 ML-KEM-768/random_bytes 正对照因 Welch $p = 0.033$ 未达 0.001 判定阈值,但分类准确率为 1.000,属于阈值边界情形)。三种环境下检测管线均得到一致的负检测结论,初步说明本研究结论具有一定跨平台和跨实现稳健性。其中 liboqs 第 1 轮的最大真实最佳平衡准确率达到 0.622,仍低于 0.65 阈值,且未与 Welch 检验和效应量阈值同时成立;因此它更适合作为附录中需要重点查看的边界样本,而不是阳性泄漏证据。

从三个证据维度看,补充实验与主实验的关系是互相印证而非完全重复。第一,分类证据上,两个后端的真实场景均值仍集中在 0.52–0.53 附近,明显低于正对照。第二,效应量证据上,个别补充实验的 $|d|$ 可超过 0.2,但没有同时伴随足够高的分组分类准确率;这说明单一效应量在软件计时噪声下仍可能被局部运行状态放大,不能脱离联合判定规则单独解释。第三,正对照证据上,71/72 个补充条件通过,唯一未通过条件仍有 1.000 的分类准确率,表明该异常更像是 Welch 阈值上的统计边界,而不是检测管线失效。完整逐条件数据见附录表 12。

表 9 macOS arm64 跨平台跨实现补充实验 (3 参数集 $\times$ 4 策略,每轮 12 次实验)

后端	轮次	均值最佳准确率	最大最佳准确率	最大 $ d $	泄漏判定	正对照通过
pqcrypto	1	0.5246	0.547	0.134	0/12	12/12
	2	0.5195	0.544	0.216	0/12	12/12
	3	0.5179	0.554	0.138	0/12	12/12
liboqs	1	0.5452	0.622	0.231	0/12	11/12*
	2	0.5266	0.567	0.139	0/12	12/12
	3	0.5328	0.566	0.274	0/12	12/12

* 该轮 ML-KEM-768/random_bytes 正对照分类准确率为 1.000,但 Welch $p = 0.033$ 未达 0.001 判定阈值,属于统计阈值边界情形。

5.7 本节证据索引

本节各子节共同构成对五个研究问题的证据链,下面指出每个问题对应的关键图表,便于读者在讨论和结束语部分将结论追溯回具体数据。

RQ1 (统计检验能否识别时间差异?) 的证据集中于图 2–3 的分布直方图和表 4 的全参数集 $\times$ 策略统计矩阵。两类密文时间分布在视觉上高度重叠;矩阵中 p 值和 $|d|$ 跨 60 组条件的分布决定了是否触发联合判定阈值。

RQ2 (机器学习分类器能否稳定区分?) 的核心证据见表 5 (四种分类器准确率汇总)、图 5 (分策略准确率对比) 和图 4 (置换检验辅助诊断)。正对照行与真实场景行的对比,是判断负结论能否归因于管线失效的关键。

RQ3 (参数集和策略是否影响结论?) 可在表 4 的行列细节中逐条核查:三个参数集与四种策略形成的 12 个交叉条件,各自的统计量和泄漏判定结果均独立列出,读者可自行比较任意子集。

RQ4 (检测灵敏度边界?) 由延迟扫描子节 (图 6、表 6) 和功效分析 (图 7、表 7) 共同刻画,给出各参数集在 80% 功效下的最小可检测效应量及对应时间量级。

RQ5 (跨平台跨实现一致性?) 的外部验证数据集中于表 9 (macOS arm64 两后端 6 轮汇总),正对照通过率和真实场景均值准确率可与主实验 Windows 数据直接对照。

方法可信度诊断 (图 8 和图 9 是否排除了实验混淆因素?) 对应第 5.5 节。图 8 验证跨 60 次实验的时间差不存在系统性漂移趋势,排除了实验间状态变化作为负结论替代解释的可能;图 9 验证代表性运行内部采集顺序与标签分配不存在可见的伪相关,排除了随机化不充分导致的顺序混淆。这两项诊断不直接回答 RQ1–RQ5,而是为上述五个研究问题的答案提供方法层面的可信度保证。

附录表 11 和 12 提供逐实验的完整明细,供需要追溯任一单次条件数据的读者参考。

6 讨论

6.1 结果解释

统计检验、效应量和机器学习结果共同支持同一结论：在当前实验设置下，未检测到 pqcrypto ML-KEM 解封装实现有效密文和四类无效密文表现出稳定、可利用的时间差异。尤其需要注意的是，真实场景的最佳模型准确率最高单次仅 0.5566，虽偶有高于 0.5 的结果，但跨轮次、跨参数集和跨策略后均不稳定，也没有同时满足统计显著性与效应量阈值。正对照的高准确率说明这一负结果不能简单归因于检测程序失效。

从数据本身看，实验结果更像是噪声条件下的弱随机波动，而不是采集失败或明显泄漏。60 次真实场景中，仅 1 次最佳平衡准确率超过 0.55，且没有任何一次超过 0.60；Welch's t 检验只有 2 次低于 0.05，没有一次低于 0.01；Cohen's d 的绝对值最大为 0.1501，低于预设的小效应阈值 0.2。平均时间差的方向也不稳定，跨实验最小值为 -1691.5 ns，最大值为 2014.8 ns，说明单次均值差异容易受到操作系统调度和运行状态影响。与之相对，正对照最佳平衡准确率均值达到 0.9995，说明当时时间信号足够强时，当前流程能够稳定识别。因此，结果应被表述为有边界的负检测结果，而不是算法或实现的形式化安全证明。

需要强调的是，上述负检测结果并不意味着检测流程缺乏灵敏度。固定延迟正对照在全部实验中均能被稳定识别，延迟灵敏度扫描也显示，当人为延迟达到一定量级时，统计检验、效应量和机器学习分类会同时给出明确响应。因此，真实场景未触发泄漏判定，更合理的解释是在当前软件计时条件和输入模型下，被测实现没有表现出超过检测阈值的稳定时间差异。

ML-KEM-512、ML-KEM-768 和 ML-KEM-1024 的平均解封装时间随参数增大而增加，single_bit 策略下有效密文 5 轮均值约为 28.25 μ s、41.96 μ s 和 60.12 μ s。但计算量增长并未带来更明显的有效/无效分类信号，说明在所测实现和输入模型下，参数集变化主要影响整体耗时，而不产生稳定标签相关差异。

表 4 中 $\Delta\mu$ 的符号模式也支持这一解释。若某一无效密文策略触发了结构性的时间偏置，通常应在不同参数集之间呈现较一致的方向；但实验中 single_bit 在 512 和 768 上为负、在 1024 上接近零且略为正，zero 在 512 上为正、在 768 和 1024 上为负。这种跨参数集符号翻转，与测量噪声和运行状态差异主导单次均值偏移的解释更一致，而不是存在一种稳定、可泛化的输入相关时间信号。

6.2 实验局限性

上述结论具有明确边界。首先，软件计时在通用操作系统上不可避免地受到任务调度、动态频率、缓存状态和后台 I/O 的影响。聚合重复、预热和随机打乱只能降低噪声，不能等同于硬件实验台的隔离测量。其次，实验的标签区分的是有效密文与无效密文，而不是私钥比特或秘密中间值；即便检测到差异，也仍需进一步分析其是否可转化为密钥恢复攻击。第三，实验对象限定为 pqcrypto==0.3.4 的具体绑定，不能直接推广至所有 ML-KEM 实现、所有编译器选项或所有处理器平台。最后，我们只测试了四类无效密文模型，不能覆盖所有选择密文输入空间。

此外，表 7 给出的 MDE 是每类 $n = 400$ 这一实验预算下的灵敏度边界，而不是检测方法本身的固有极限。由式 (10) 可知，最小可检测效应量随样本量近似按 $1/\sqrt{n}$ 缩小；若显著增加查询次数并进一步降低系统噪声，原则上可以探测远小于当前边界的时间差异。因此，结论应表述为：在与本研究实验预算和软件计时条件相当的观测规模下，未发现超过约 2.4–3.5 μ s 灵敏度边界的稳定均值差异；更大预算下是否仍无可分辨信号，需要相应规模的重复实验来验证。

关于平台与实现局限性，第 5.6 节给出的 macOS arm64 与 liboqs 结果只是稳健性检查，能够说明相同流程在另外两个环境下没有给出相反信号，但不能替代系统性的跨操作系统、跨编译器和跨微架构评估。

此外，本研究的观测粒度是端到端解封装耗时，而 KyberSlash 等工作揭示的密钥相关时序泄漏可能来自单条变长除法或压缩指令，其耗时差异在纳秒级总时间中可能被其他噪声源淹没^[16]。因此，上述负结果不能说明被测实现在指令级别不存在此类问题，只能说明在端到端黑盒计时这一观测粒度下未发现稳定可区分信号；若要排查 KyberSlash 类问题，需要结合源码审计或指令级污点分析等更细粒度的方法。

6.3 可复现性与后续工作

项目保留了完整采集脚本、原始 CSV、JSON 分析报告、数据质量报告和论文图表。后续工作可以从三个方向展开：第一，在 Linux x86_64 和不同 CPU 微架构上完成更大规模的跨平台矩阵（macOS arm64 的 pqcrypto/liboqs 补充实验已在本研究中给出初步结果）；第二，使用更高重复次数或进程绑定、性能模式锁定等方法降低软件计时噪声；第三，将同样的输入模型扩展到功耗或电磁轨迹，比较软件时序与物理侧信道的灵敏度差异。

为支持独立复现，采集脚本与图表生成代码随仓库提供；原始计时 CSV、统计与机器学习分析的 JSON 报告、数据质量报告以及生成本研究图表所需的实验输出随项目 Release 数据包一并提供，具体目录结构和重新运行命令见附录。第三方可在安装相同版本依赖后，直接基于附录中的命令重新生成本研究主要表图，包括表 4-8 和图 2-9 中的全部结果。

7 结束语

本研究构建并完整执行了一套针对 ML-KEM 解封装实现的软件时间侧信道筛查框架。相比于孤立的统计检验，该框架的方法论价值体现在三点：联合判定准则（统计显著性 + 效应量 + 分类准确率）防止单一指标的偶然触发；固定延迟正对照将“管线是否有效”与“实现是否泄漏”解耦，使负结论具备可证伪性；延迟灵敏度扫描将结论边界量化为可解读的时间量级，而非笼统断言“未发现泄漏”。

在 5 轮 $\times$ 3 参数集 $\times$ 4 无效密文策略共 60 次主实验中，统计检验与机器学习分类均未发现稳定可区分的时间差异。有效密文与无效密文的时间差均值仅 81.6 ns，却淹没在标准差 1044.9 ns 的噪声之中；最大单次 $|d|$ 为 0.150，低于小效应阈值 0.2，Welch p 值无一低于 0.01；四种分类器最佳平衡准确率均值 0.5134、最高单次 0.5566，始终接近随机基线 0.5。与此形成对照的是，相同框架下正对照均值达 0.9995，表明管线本身有效，负结论不是检测失灵的产物。三个参数集与四种无效密文策略（单比特扰动至全零密文）的 12 个交叉条件均独立给出相同负结论，说明结果与具体的密文构造方式和参数集选择无关。

延迟灵敏度扫描将这一负结论的可信边界量化为 80% 功效下约 $2.4 \mu\text{s}$ (ML-KEM-512) 至 $3.5 \mu\text{s}$ (ML-KEM-1024)。这意味着若被测实现存在亚微秒量级的指令级时间差异（如 KyberSlash 所揭示的变长除法问题），该差异仍可能被软件计时噪声掩盖而不在当前观测粒度下成为稳定信号，这是本研究结论需要明确说明的边界。在 macOS arm64 平台 `pqcrypto` 和 `liboqs` 两种后端各 3 轮共 72 次补充实验中，正对照 71/72 通过，两种后端真实场景均值最佳平衡准确率分别为 0.521 和 0.535，与主实验结论高度一致，初步排除了结果依赖特定平台或绑定实现的可能。

上述结果对需要快速评估 ML-KEM 库函数部署风险的开发者具有直接参考价值：在当前软件黑盒计时条件下，被测实现目前不表现出可稳定利用的端到端时间差异；但指令级泄漏仍超出软件时序筛查的分辨范围，需要硬件性能计数器、功耗分析或源码级污点追踪才能进一步排查。

本研究验证了一种可复现的筛查思路：以正对照确认管线灵敏度、以多轮重复稳定结论、以 MDE 量化边界、以跨平台重复检验一致性。这一流程可以推广到其他后量子密码实现的初步安全评估中。未来工作可以在硬件性能计数器、功耗分析和指令级源码审计等更细粒度的工具上推进，以更完整地回答 ML-KEM 实现的侧信道安全问题。

参考文献

- [1] SHOR P W. Algorithms for Quantum Computation: Discrete Logarithms and Factoring//Proceedings 35th Annual Symposium on Foundations of Computer Science. 1994: 124-134. DOI: [10.1109/SFCS.1994.365700](https://doi.org/10.1109/SFCS.1994.365700).
- [2] National Institute of Standards and Technology Module-Lattice-Based Key-Encapsulation Mechanism Standard. Federal Information Processing Standard FIPS 203. Gaithersburg, MD, 2024. DOI: [10.6028/NIST.FIPS.203](https://doi.org/10.6028/NIST.FIPS.203).
- [3] AVANZI R, BOS J, DUCAS L, CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation, Version 3.02. Round 3 submission to the NIST Post-Quantum Cryptography Standardization Project. 2021. <https://pq-crystals.org/kyber/>.
- [4] KOCHER P C. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems//Lecture Notes in Computer Science: Advances in Cryptology -- CRYPTO 1996: vol. 1109. Springer, 1996: 104-113. DOI: [10.1007/3-540-68697-5_9](https://doi.org/10.1007/3-540-68697-5_9).
- [5] BERNSTEIN D J. Cache-Timing Attacks on AES. 2005. <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>.
- [6] RAVI P, ROY S S, CHATTOPADHYAY A. Generic Side-channel Attacks on CCA-secure Lattice-based PKE and KEM Schemes//IACR Transactions on Cryptographic Hardware and Embedded Systems: vol. 2020: 3. 2020: 307-335. DOI: [10.13154/tches.v2020.i3.307-335](https://doi.org/10.13154/tches.v2020.i3.307-335).
- [7] UENO R, XAGAWA K, TANAKA Y. Curse of Re-encryption: A Generic Power/EM Analysis on Post-quantum KEMs//IACR Transactions on Cryptographic Hardware and Embedded Systems: vol. 2022: 1. 2022: 296-322. DOI: [10.46586/tches.v2022.i1.296-322](https://doi.org/10.46586/tches.v2022.i1.296-322).
- [8] BECKER G T, COOPER J, DEMULDER E. Test Vector Leakage Assessment (TVLA) Methodology in Practice//International Cryptographic Module Conference. 2013.
- [9] KANNWISCHER M J, RIJNEVELD J, SCHWABE P. pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4//IACR Transactions on Cryptographic Hardware and Embedded Systems: vol. 2019: 2. 2019: 1-39. DOI: [10.13154/tches.v2019.i2.1-39](https://doi.org/10.13154/tches.v2019.i2.1-39).
- [10] BOS J W, DUCAS L, KILTZ E, CRYSTALS -- Kyber: A CCA-Secure Module-Lattice-Based KEM//2018 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2018: 353-367. DOI: [10.1109/EuroSP.2018.00032](https://doi.org/10.1109/EuroSP.2018.00032).
- [11] BRUMLEY D, BONEH D. Remote Timing Attacks are Practical. Computer Networks, 2005, 48(5): 701-716. DOI: [10.1016/j.comnet.2005.01.010](https://doi.org/10.1016/j.comnet.2005.01.010).
- [12] WELCH B L. The Generalization of 'Student's' Problem when Several Different Population Variances are Involved. Biometrika, 1947, 34(1/2): 28-35. DOI: [10.2307/2332510](https://doi.org/10.2307/2332510).
- [13] COHEN J. Statistical Power Analysis for the Behavioral Sciences. 2nd. Lawrence Erlbaum Associates, 1988.
- [14] MANN H B, WHITNEY D R. On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other. The Annals of Mathematical Statistics, 1947, 18(1): 50-60. DOI: [10.1214/aoms/1177730491](https://doi.org/10.1214/aoms/1177730491).
- [15] PRIMAS R, PESSL P, MANGARD S. Single-Trace Side-Channel Attacks on Masked Lattice-Based Encryption//Lecture Notes in Computer Science: Cryptographic Hardware and Embedded Systems -- CHES 2017: vol. 10529. Springer, 2017: 513-533. DOI: [10.1007/978-3-319-66787-4_25](https://doi.org/10.1007/978-3-319-66787-4_25).
- [16] BERNSTEIN D J, BHARGAVAN K, BHASIN S. KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2025, 2025(2): 209-234. DOI: [10.46586/tches.v2025.i2.209-234](https://doi.org/10.46586/tches.v2025.i2.209-234).
- [17] LERMAN L, BONTEMPI G, MARKOWITZ O. Power Analysis Attack: An Approach Based on Machine Learning. International Journal of Applied Cryptography, 2015, 3(2): 97-115. DOI: [10.1504/IJACT.2015.071829](https://doi.org/10.1504/IJACT.2015.071829).
- [18] CAGLI E, DUMAS C, PROUFF E. Convolutional Neural Networks with Data Augmentation Against Jitter-Based Countermeasures//Lecture Notes in Computer Science: Cryptographic Hardware and Embedded Systems -- CHES 2017: vol. 10529. Springer, 2017: 45-68. DOI: [10.1007/978-3-319-66787-4_3](https://doi.org/10.1007/978-3-319-66787-4_3).
- [19] REGEV O. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography//Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing. ACM, 2005: 84-93. DOI: [10.1145/1060590.1060603](https://doi.org/10.1145/1060590.1060603).
- [20] GROVER L K. A Fast Quantum Mechanical Algorithm for Database Search//Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. ACM, 1996: 212-219. DOI: [10.1145/237814.237866](https://doi.org/10.1145/237814.237866).
- [21] BREIMAN L. Random Forests. Machine Learning, 2001, 45(1): 5-32. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [22] FRIEDMAN J H. Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 2001, 29(5): 1189-1232. DOI: [10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451).
- [23] PEDREGOSA F, VAROQUAUX G, GRAMFORT A. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011, 12: 2825-2830. <https://jmlr.org/papers/v12/pedregosa11a.html>.

附录

实验复现命令

实验代码已保存在项目目录中。完整实验可在安装依赖后通过如下命令运行：

```
python -m pip install -r requirements.txt
python -m pip install -r requirements-dev.txt
python -m pip install -e .
export LOKY_MAX_CPU_COUNT=8
export MLKEM_PERMUTATIONS=20
bash scripts/run_paper_experiments.sh
```

其中，MLKEM_PERMUTATIONS 只影响输出报告中的置换检验辅助诊断，不参与正文泄漏判定规则；若需要更精细的置换检验 p 值，可提高该参数并重新运行分析。
若只需快速检查环境，可使用小规模冒烟测试：

```
RUNS=1 SAMPLES_PER_CLASS=20 REPETITIONS=5 N_GROUPS=5 \
VARIANTS="768" INVALID_STRATEGIES="single_bit" \
MLKEM_PERMUTATIONS=5 bash scripts/run_paper_experiments.sh
```

主要数据文件说明

论文实验相关输出文件如表 10 所示。

表 10 论文实验相关输出文件

路径	说明
results/paper_runs/	5 轮完整实验的原始 CSV 与 JSON 分析结果
results/paper_artifacts/	论文图表与数据质量报告
results/paper_artifacts/DATA_QUALITY_REPORT.md	标签平衡、缺失值、正对照有效性等检查结果
summary.json	每个参数集/策略组合的统计检验和模型结果
results/compare5_pqcrypto/	macOS arm64/pqcrypto 跨平台补充实验（3 轮）
results/compare5_liboqs/	macOS arm64/liboqs 跨实现补充实验（3 轮）

泄漏判定规则

我们采用保守的联合判定方式。真实场景只有在以下三个条件同时满足时，才报告检测到时间泄漏：最佳模型平衡准确率不低于 0.65；Welch’s t 检验 $p < 0.01$ ；Cohen’s d 的绝对值不低于 0.2。正对照要求最佳模型平衡准确率不低于 0.90，且 Welch’s t 检验 $p < 0.001$ 。这一规则用于避免单一统计量偶然显著导致误判。

逐轮完整实验结果（60 次，Windows x86_64/pqcrypto）

Windows x86_64 平台上 60 次完整实验的逐轮结果如表 11 所示。

表 11 60 次完整实验逐轮结果（Windows x86_64, pqcrypto==0.3.4）

轮次	参数集	策略	$\Delta\mu$ (ns)	Welch p	Cohen’s d	最佳平衡准确率
1	ML-KEM-512	byte_flip	+656.2	0.100	+0.117	0.509
1	ML-KEM-512	random_bytes	−1298.6	0.295	−0.074	0.516

续下页

轮次	参数集	策略	$\Delta\mu$ (ns)	Welch p	Cohen's d	最佳平衡准确率
1	ML-KEM-512	single_bit	-208.2	0.527	-0.045	0.534
1	ML-KEM-512	zero	+1238.5	0.315	+0.071	0.536
1	ML-KEM-768	byte_flip	+1284.7	0.356	+0.065	0.526
1	ML-KEM-768	random_bytes	-270.0	0.112	-0.113	0.508
1	ML-KEM-768	single_bit	-1291.2	0.302	-0.073	0.499
1	ML-KEM-768	zero	-1168.0	0.359	-0.065	0.506
1	ML-KEM-1024	byte_flip	+1252.4	0.336	+0.068	0.500
1	ML-KEM-1024	random_bytes	+2014.8	0.131	+0.107	0.510
1	ML-KEM-1024	single_bit	-55.6	0.841	-0.014	0.519
1	ML-KEM-1024	zero	-1555.0	0.236	-0.084	0.506
2	ML-KEM-512	byte_flip	+905.3	0.434	+0.055	0.502
2	ML-KEM-512	random_bytes	-50.8	0.708	-0.027	0.515
2	ML-KEM-512	single_bit	+243.6	0.034	+0.150	0.517
2	ML-KEM-512	zero	+1283.5	0.301	+0.073	0.519
2	ML-KEM-768	byte_flip	-122.3	0.517	-0.046	0.504
2	ML-KEM-768	random_bytes	+47.8	0.776	+0.020	0.503
2	ML-KEM-768	single_bit	-1126.1	0.351	-0.066	0.501
2	ML-KEM-768	zero	-1500.1	0.264	-0.079	0.506
2	ML-KEM-1024	byte_flip	-35.8	0.876	-0.011	0.494
2	ML-KEM-1024	random_bytes	-153.6	0.478	-0.050	0.513
2	ML-KEM-1024	single_bit	+303.0	0.274	+0.077	0.525
2	ML-KEM-1024	zero	-1691.5	0.255	-0.081	0.527
3	ML-KEM-512	byte_flip	+158.1	0.204	+0.090	0.527
3	ML-KEM-512	random_bytes	+1380.9	0.309	+0.072	0.493
3	ML-KEM-512	single_bit	-228.9	0.039	-0.147	0.513
3	ML-KEM-512	zero	+1290.4	0.326	+0.070	0.521
3	ML-KEM-768	byte_flip	+1277.4	0.396	+0.060	0.497
3	ML-KEM-768	random_bytes	+4.0	0.982	+0.002	0.517
3	ML-KEM-768	single_bit	+1336.4	0.268	+0.078	0.504
3	ML-KEM-768	zero	-1069.8	0.413	-0.058	0.529
3	ML-KEM-1024	byte_flip	+727.9	0.559	+0.041	0.519
3	ML-KEM-1024	random_bytes	-216.1	0.559	-0.041	0.527
3	ML-KEM-1024	single_bit	+1088.5	0.395	+0.060	0.490
3	ML-KEM-1024	zero	+1645.5	0.209	+0.089	0.529
4	ML-KEM-512	byte_flip	-1025.4	0.344	-0.067	0.512
4	ML-KEM-512	random_bytes	+104.3	0.449	+0.054	0.487
4	ML-KEM-512	single_bit	-82.8	0.486	-0.049	0.504
4	ML-KEM-512	zero	-104.0	0.381	-0.062	0.530
4	ML-KEM-768	byte_flip	-767.2	0.533	-0.044	0.523
4	ML-KEM-768	random_bytes	+1051.3	0.389	+0.061	0.505
4	ML-KEM-768	single_bit	-1394.6	0.237	-0.084	0.523
4	ML-KEM-768	zero	+1358.7	0.306	+0.072	0.508
4	ML-KEM-1024	byte_flip	-28.2	0.894	-0.009	0.495
4	ML-KEM-1024	random_bytes	+1627.1	0.234	+0.084	0.503
4	ML-KEM-1024	single_bit	-266.9	0.263	-0.079	0.520
4	ML-KEM-1024	zero	-203.2	0.263	-0.079	0.545
5	ML-KEM-512	byte_flip	-1109.8	0.335	-0.068	0.536
5	ML-KEM-512	random_bytes	-1189.8	0.378	-0.062	0.520
5	ML-KEM-512	single_bit	+120.6	0.291	+0.075	0.505
5	ML-KEM-512	zero	+1234.7	0.312	+0.072	0.531
5	ML-KEM-768	byte_flip	+1511.5	0.279	+0.077	0.504
5	ML-KEM-768	random_bytes	+54.8	0.781	+0.020	0.522
5	ML-KEM-768	single_bit	-1426.6	0.263	-0.079	0.498
5	ML-KEM-768	zero	+1249.8	0.320	+0.070	0.476
5	ML-KEM-1024	byte_flip	+1684.7	0.225	+0.086	0.522
5	ML-KEM-1024	random_bytes	-1287.0	0.385	-0.062	0.513
5	ML-KEM-1024	single_bit	-1038.5	0.434	-0.055	0.557

续下页

轮次	参数集	策略	$\Delta\mu$ (ns)	Welch p	Cohen's d	最佳平衡准确率
5	ML-KEM-1024	zero	-1277.2	0.349	-0.066	0.508

逐轮补充实验结果 (72 次, macOS arm64/pqcrypto 与 liboqs)

表 12 给出跨平台与跨实现补充实验的逐次摘要。正文第 5.6 节只汇总到后端和轮次层面；本表保留每个后端、轮次、参数集和无效密文策略的真实场景核心指标。72 次补充实验均未触发泄漏判定，表中“正对照通过”只表示固定延迟正对照是否同时满足准确率与 Welch p 阈值。

表 12 macOS arm64 补充实验逐轮结果 (pqcrypto 与 liboqs)

后端	轮次	参数集	策略	$\Delta\mu$ (ns)	Welch p	Cohen's d	最佳平衡准确率	正对照通过
pqcrypto	1	ML-KEM-512	single_bit	+104.0	0.172	+0.097	0.547	是
pqcrypto	1	ML-KEM-512	byte_flip	-1614.0	0.313	-0.071	0.519	是
pqcrypto	1	ML-KEM-512	random_bytes	-1334.4	0.274	-0.077	0.511	是
pqcrypto	1	ML-KEM-512	zero	-1159.9	0.225	-0.086	0.513	是
pqcrypto	1	ML-KEM-768	single_bit	-1749.3	0.145	-0.103	0.524	是
pqcrypto	1	ML-KEM-768	byte_flip	-1532.7	0.188	-0.093	0.545	是
pqcrypto	1	ML-KEM-768	random_bytes	+824.8	0.440	+0.055	0.522	是
pqcrypto	1	ML-KEM-768	zero	-243.8	0.059	-0.134	0.522	是
pqcrypto	1	ML-KEM-1024	single_bit	-295.4	0.103	-0.116	0.521	是
pqcrypto	1	ML-KEM-1024	byte_flip	-272.4	0.078	-0.125	0.503	是
pqcrypto	1	ML-KEM-1024	random_bytes	+604.9	0.599	+0.037	0.524	是
pqcrypto	1	ML-KEM-1024	zero	-1744.0	0.178	-0.095	0.544	是
pqcrypto	2	ML-KEM-512	single_bit	+25.0	0.686	+0.040	0.535	是
pqcrypto	2	ML-KEM-512	byte_flip	+49.9	0.537	+0.062	0.497	是
pqcrypto	2	ML-KEM-512	random_bytes	+42.3	0.825	+0.022	0.536	是
pqcrypto	2	ML-KEM-512	zero	+372.7	0.031	+0.216	0.544	是
pqcrypto	2	ML-KEM-768	single_bit	+173.1	0.499	+0.068	0.512	是
pqcrypto	2	ML-KEM-768	byte_flip	+292.9	0.129	+0.152	0.505	是
pqcrypto	2	ML-KEM-768	random_bytes	+145.8	0.619	+0.050	0.527	是
pqcrypto	2	ML-KEM-768	zero	+54.5	0.820	+0.023	0.532	是
pqcrypto	2	ML-KEM-1024	single_bit	-191.1	0.393	-0.085	0.518	是
pqcrypto	2	ML-KEM-1024	byte_flip	+663.4	0.208	+0.126	0.528	是
pqcrypto	2	ML-KEM-1024	random_bytes	+1400.6	0.250	+0.115	0.497	是
pqcrypto	2	ML-KEM-1024	zero	+54.2	0.722	+0.036	0.502	是
pqcrypto	3	ML-KEM-512	single_bit	+17.6	0.622	+0.049	0.514	是
pqcrypto	3	ML-KEM-512	byte_flip	+68.3	0.387	+0.087	0.524	是
pqcrypto	3	ML-KEM-512	random_bytes	+84.8	0.736	+0.034	0.487	是
pqcrypto	3	ML-KEM-512	zero	+105.5	0.585	+0.055	0.530	是
pqcrypto	3	ML-KEM-768	single_bit	+321.0	0.508	+0.066	0.524	是
pqcrypto	3	ML-KEM-768	byte_flip	-19.6	0.935	-0.008	0.554	是
pqcrypto	3	ML-KEM-768	random_bytes	-50.4	0.816	-0.023	0.496	是
pqcrypto	3	ML-KEM-768	zero	-6136.0	0.321	-0.100	0.496	是
pqcrypto	3	ML-KEM-1024	single_bit	+162.4	0.638	+0.047	0.514	是
pqcrypto	3	ML-KEM-1024	byte_flip	+577.0	0.169	+0.138	0.521	是
pqcrypto	3	ML-KEM-1024	random_bytes	+524.7	0.176	+0.136	0.537	是
pqcrypto	3	ML-KEM-1024	zero	-103.6	0.901	-0.013	0.518	是
liboqs	1	ML-KEM-512	single_bit	-69.5	0.317	-0.100	0.579	是
liboqs	1	ML-KEM-512	byte_flip	-96.2	0.278	-0.109	0.530	是
liboqs	1	ML-KEM-512	random_bytes	-277.0	0.180	-0.135	0.523	是
liboqs	1	ML-KEM-512	zero	-45.3	0.562	-0.058	0.531	是
liboqs	1	ML-KEM-768	single_bit	-32.3	0.279	-0.108	0.492	是
liboqs	1	ML-KEM-768	byte_flip	+133.8	0.211	+0.125	0.572	是
liboqs	1	ML-KEM-768	random_bytes	+61.9	0.495	+0.068	0.521	否
liboqs	1	ML-KEM-768	zero	-6.8	0.907	-0.012	0.526	是
liboqs	1	ML-KEM-1024	single_bit	-164.8	0.021	-0.231	0.567	是
liboqs	1	ML-KEM-1024	byte_flip	-26.8	0.713	-0.037	0.514	是
liboqs	1	ML-KEM-1024	random_bytes	-121.5	0.169	-0.138	0.622	是
liboqs	1	ML-KEM-1024	zero	-11.9	0.862	-0.017	0.567	是
liboqs	2	ML-KEM-512	single_bit	-16.9	0.374	-0.089	0.535	是
liboqs	2	ML-KEM-512	byte_flip	+64.0	0.222	+0.122	0.567	是
liboqs	2	ML-KEM-512	random_bytes	+52.3	0.228	+0.121	0.547	是
liboqs	2	ML-KEM-512	zero	-16.9	0.711	-0.037	0.485	是
liboqs	2	ML-KEM-768	single_bit	+17.1	0.899	+0.013	0.545	是
liboqs	2	ML-KEM-768	byte_flip	+155.0	0.337	+0.096	0.499	是

续下页

后端	轮次	参数集	策略	$\Delta\mu$ (ns)	Welch p	Cohen's d	最佳平衡准确率	正对照通过
liboqs	2	ML-KEM-768	random_bytes	+46.4	0.778	+0.028	0.508	是
liboqs	2	ML-KEM-768	zero	-100.2	0.427	-0.079	0.511	是
liboqs	2	ML-KEM-1024	single_bit	-39.1	0.659	-0.044	0.543	是
liboqs	2	ML-KEM-1024	byte_flip	+240.9	0.166	+0.139	0.511	是
liboqs	2	ML-KEM-1024	random_bytes	+173.8	0.327	+0.098	0.564	是
liboqs	2	ML-KEM-1024	zero	-150.3	0.373	-0.089	0.504	是
liboqs	3	ML-KEM-512	single_bit	-106.8	0.353	-0.093	0.550	是
liboqs	3	ML-KEM-512	byte_flip	-128.3	0.688	-0.040	0.566	是
liboqs	3	ML-KEM-512	random_bytes	+4.8	0.893	+0.013	0.538	是
liboqs	3	ML-KEM-512	zero	+52.4	0.192	+0.131	0.543	是
liboqs	3	ML-KEM-768	single_bit	+0.5	0.989	+0.001	0.539	是
liboqs	3	ML-KEM-768	byte_flip	+308.2	0.006	+0.274	0.547	是
liboqs	3	ML-KEM-768	random_bytes	-145.9	0.184	-0.133	0.545	是
liboqs	3	ML-KEM-768	zero	+59.5	0.658	+0.044	0.515	是
liboqs	3	ML-KEM-1024	single_bit	+215.1	0.249	+0.116	0.514	是
liboqs	3	ML-KEM-1024	byte_flip	+8025.2	0.312	+0.101	0.484	是
liboqs	3	ML-KEM-1024	random_bytes	+161.2	0.397	+0.085	0.547	是
liboqs	3	ML-KEM-1024	zero	+78.2	0.676	+0.042	0.508	是